Doc. no.	P0913R0
Date:	2018-02-08 at 14:25:00 UTC
Reference:	ISO/IEC TS 22277, C++ Extensions for Coroutines
Audience:	EWG
Reply to:	Gor Nishanov <gorn@microsoft.com>

Add symmetric coroutine control transfer

Issue

Currently Coroutines TS only supports asymmetric control transfer where suspend always returns control back to the current coroutine caller or resumer. In order to emulate symmetric coroutine to coroutine control transfer, one needs to build a queue and a scheduler: prior to suspension, a coroutine enqueues the next-to-resume coroutine into a scheduler queue and then returns control to its caller. At some point, control will transfer to a scheduler loop that dequeues and resumes coroutines.

Recursive generators, zero-overhead futures and other facilities require efficient coroutine to coroutine control transfer. Involving a queue and a scheduler makes coroutine to coroutine control transfer inefficient. Coroutines need direct and efficient way of expressing the desired behavior.

Proposed resolution:

1. Allow `await_suspend` to designate a coroutine to perform a symmetric control transfer to.
2. Add library function `noop_coroutine` that returns a handle to a coroutine that has no observable side effects when resumed. Having such a coroutine handle allows library writer to perform either symmetric or asymmetric control transfer based on runtime considerations.

Before

```
thread_local queue<coroutine_handle<>> resume_queue;

void scheduler_loop() {
  while (!resume_queue.empty()) {
    resume_queue.front().resume();
    resume_queue.pop_front();
  }
}

struct Awaiter {
  ...
  void await_suspend(coroutine_handle<> h) {
    ...
    if (cond)
      resume_queue.push(next_coro);
  }
};
```

After

```
struct Awaiter {
  ...
  auto await_suspend(coroutine_handle<> h) {
    ...
    return cond ? next_coro : noop_coroutine();
  }
};
```

Implementation and usage experience:

Language changes implement in clang 5.1. Adopted by open source coroutine class library cppcoro.

Wording

[All proposed wording is relative to N4680 (ISO/IEC TS 22277)].

Core wording:

Modify paragraph 5.3.8/3 as follows

> (3.7) — await-suspend is the expression e.`await_suspend(h)`, which shall be a prvalue of type `void` ,~~or~~ `bool`, or `std::experimental::coroutine_handle<Z>` for some type `Z` .

Modify paragraph 5.3.8/5 as follows:

> 5 The await-expression evaluates the await-ready expression, then:
>
> (5.1) — If the result is `false`, the coroutine is considered suspended. Then, the *await-suspend* expression is evaluated. If that expression has type `std::experimental::coroutine_handle<Z>` and evaluates to value *s*, the coroutine referred to by *s* is resumed as if by a call *s*`.resume()` and then control flow returns to the current coroutine caller or resumer (8.4.4). Implementations shall not impose any limits on how many coroutines can be resumed in this fashion. If that expression has type `bool` and evaluates to `false`, the coroutine is resumed. If that expression exits via an exception, the exception is caught, the coroutine is resumed, and the exception is immediately re-thrown (15.1). Otherwise, control flow returns to the current coroutine caller or resumer (8.4.4) ~~without exiting any scopes (6.6)~~.

Library Wording:

Add the following to `<experimental/coroutine>` synopsis in [support.coroutine]/1:

```
namespace std {
namespace experimental {
inline namespace coroutines_v1 {

// class noop_coroutine_promise 18.11.4
struct noop_coroutine_promise;

// 18.11.1 coroutine traits
template <typename R, typename... ArgTypes>
struct coroutine_traits;

// 18.11.2 coroutine handle
template <typename Promise = void>
struct coroutine_handle;

template <>
struct coroutine_handle<noop_coroutine_promise>;

// noop_coroutine_handle 18.11.5
using noop_coroutine_handle = coroutine_handle<noop_coroutine_promise>;

// noop_coroutine 18.11.6
noop_coroutine_handle noop_coroutine() noexcept;

...
```

Add the following to `coroutine_handle` synopsis in [coroutine.handle]:

```
template <> struct coroutine_handle<noop_coroutine_promise> : coroutine_handle<>
{
// 18.11.2.2 export
constexpr void* address() const noexcept;

// 18.11.2.7 noop observers
constexpr explicit operator bool() const noexcept;
constexpr bool done() const noexcept;

// 18.11.2.8 noop resumption
constexpr void operator()() const noexcept;
constexpr void resume() const noexcept;
constexpr void destroy() const noexcept;

// 18.11.2.9 noop promise access
noop_coroutine_promise& promise() const noexcept;
private:
```

```
// constructor unspecified
};
```

Modify subclause 18.11.2.2 [coroutine.handle.export.import] as follows:

18.11.2.2 `coroutine_handle` export/import [coroutine.handle.export.import]

```
constexpr void* address() const noexcept;
```

1 *Returns:* `ptr`.
2 *Remarks:* A noop coroutine's `ptr` always contains non-null pointer value.

Add subclause 18.11.2.7 [coroutine.handle.noop.observers]:

18.11.2.7 `noop_coroutine_handle` observers [coroutine.handle.noop.observers]:

```
constexpr explicit operator bool() const noexcept;
```

1 *Returns:* `true`

```
constexpr bool done() const noexcept;
```

2 *Returns:* `false`

Add subclause 18.11.2.8 [coroutine.handle.noop.resumption]:

18.11.2.8 `noop_coroutine_handle` resumption [coroutine.handle.noop.resumption]:

```
constexpr void operator()() const noexcept;
constexpr void resume() const noexcept;
constexpr void destroy() const noexcept;
```

1 *Effects:* No observable side effects.
2 *Remarks:* If `noop_coroutine_handle` is converted to `coroutine_handle<>`, calls to `operator()`, `resume` and `destroy` on that handle will also have no observable side effects.

Add subclause 18.11.2.9 [coroutine.handle.noop.promise]:

18.11.2.9 `noop_coroutine_handle` promise access [coroutine.handle.noop.promise]:

```
noop_coroutine_promise& promise() const noexcept;
```

1 *Returns:* a reference to a promise object associated with this coroutine handle.

Add subclause 18.11.4 [coroutine.promise.noop]:

18.11.4 Class `noop_coroutine_promise` [coroutine.promise.noop]

```
struct noop_coroutine_promise{};
```

1 The class `noop_coroutine_promise` defines the promise type for the coroutine referred to by `noop_coroutine_handle` (18.11.5).

Add subclause 18.11.5 [coroutine.handle.noop]:

18.11.5 Class `noop_coroutine_handle` [coroutine.handle.noop]

1 The type `noop_coroutine_handle` is a synonym for `coroutine_handle<noop_coroutine_promise>`.

Add subclause 18.11.6 [coroutine.noop]:

18.11.6 Function `noop_coroutine` [coroutine.noop]

```
noop_coroutine_handle noop_coroutine() noexcept;
```

1 *Returns:* A handle to a coroutine that has no observable side effects when resumed or destroyed.

2 *Remarks:* A handle returned from `noop_coroutine` may or may not compare equal to a handle returned from another invocation of `noop_coroutine`.